

动手试一试

10-3 访客：编写一个程序，提示用户输入其名字；用户作出响应后，将其名字写入到文件 `guest.txt` 中。

10-4 访客名单：编写一个 `while` 循环，提示用户输入其名字。用户输入其名字后，在屏幕上打印一句问候语，并将一条访问记录添加到文件 `guest_book.txt` 中。确保这个文件中的每条记录都独占一行。

10-5 关于编程的调查：编写一个 `while` 循环，询问用户为何喜欢编程。每当用户输入一个原因后，都将其添加到一个存储所有原因的文件中。

10.3 异常

Python使用被称为异常的特殊对象来管理程序执行期间发生的错误。每当发生让Python不知所措的错误时，它都会创建一个异常对象。如果你编写了处理该异常的代码，程序将继续运行；如果你未对异常进行处理，程序将停止，并显示一个`traceback`，其中包含有关异常的报告。

异常是使用`try-except`代码块处理的。`try-except`代码块让Python执行指定的操作，同时告诉Python发生异常时怎么办。使用了`try-except`代码块时，即便出现异常，程序也将继续运行：显示你编写的友好的错误消息，而不是令用户迷惑的`traceback`。

10.3.1 处理 `ZeroDivisionError` 异常

下面来看一种导致Python引发异常的简单错误。你可能知道不能将一个数字除以0，但我们还是让Python这样做吧：

```
division.py
```

```
print(5/0)
```

显然，Python无法这样做，因此你将看到一个`traceback`：

```
Traceback (most recent call last):  
  File "division.py", line 1, in <module>  
    print(5/0)
```

```
❶ ZeroDivisionError: division by zero
```

在上述`traceback`中，❶处指出的错误`ZeroDivisionError`是一个异常对象。Python无法按你的要求做时，就会创建这种对象。在这种情况下，Python将停止运行程序，并指出引发了哪种异常，而我们可根据这些信息对程序进行修改。下面我们将告诉Python，发生这种错误时怎么办；这样，如果再次发生这样的错误，我们就有备无患了。

10.3.2 使用 try-except 代码块

当你认为可能发生了错误时，可编写一个try-except代码块来处理可能引发的异常。你让Python尝试运行一些代码，并告诉它如果这些代码引发了指定的异常，该怎么办。

处理ZeroDivisionError异常的try-except代码块类似于下面这样：

```
try:
    print(5/0)
except ZeroDivisionError:
    print("You can't divide by zero!")
```

我们将导致错误的代码行print(5/0)放在了一个try代码块中。如果try代码块中的代码运行起来没有问题，Python将跳过except代码块；如果try代码块中的代码导致了错误，Python将查找这样的except代码块，并运行其中的代码，即其中指定的错误与引发的错误相同。

在这个示例中，try代码块中的代码引发了ZeroDivisionError异常，因此Python指出了该如何解决问题的except代码块，并运行其中的代码。这样，用户看到的是一条友好的错误消息，而不是traceback：

```
You can't divide by zero!
```

如果try-except代码块后面还有其他代码，程序将接着运行，因为已经告诉了Python如何处理这种错误。下面来看一个捕获错误后程序将继续运行的示例。

10

10.3.3 使用异常避免崩溃

发生错误时，如果程序还有工作没有完成，妥善地处理错误就尤其重要。这种情况经常会出现现在要求用户提供输入的程序中；如果程序能够妥善地处理无效输入，就能再提示用户提供有效输入，而不至于崩溃。

下面来创建一个只执行除法运算的简单计算器：

division.py

```
print("Give me two numbers, and I'll divide them.")
print("Enter 'q' to quit.")

while True:
    ❶ first_number = input("\nFirst number: ")
    if first_number == 'q':
        break
    ❷ second_number = input("Second number: ")
    if second_number == 'q':
        break
    ❸ answer = int(first_number) / int(second_number)
    print(answer)
```

在❶处，这个程序提示用户输入一个数字，并将其存储到变量`first_number`中；如果用户输入的不是表示退出的`q`，就再提示用户输入一个数字，并将其存储到变量`second_number`中（见❷）。接下来，我们计算这两个数字的商（即`answer`，见❸）。这个程序没有采取任何处理错误的措施，因此让它执行除数为0的除法运算时，它将崩溃：

```
Give me two numbers, and I'll divide them.
Enter 'q' to quit.

First number: 5
Second number: 0
Traceback (most recent call last):
  File "division.py", line 9, in <module>
    answer = int(first_number) / int(second_number)
ZeroDivisionError: division by zero
```

程序崩溃可不好，但让用户看到`traceback`也不是好主意。不懂技术的用户会被它们搞糊涂，而且如果用户怀有恶意，他会通过`traceback`获悉你不希望他知道的信息。例如，他将知道你的程序文件的名称，还将看到部分不能正确运行的代码。有时候，训练有素的攻击者可根据这些信息判断出可对你的代码发起什么样的攻击。

10.3.4 else 代码块

通过将可能引发错误的代码放在`try-except`代码块中，可提高这个程序抵御错误的能力。错误是执行除法运算的代码行导致的，因此我们需要将它放到`try-except`代码块中。这个示例还包含一个`else`代码块；依赖于`try`代码块成功执行的代码都应放到`else`代码块中：

```
print("Give me two numbers, and I'll divide them.")
print("Enter 'q' to quit.")

while True:
    first_number = input("\nFirst number: ")
    if first_number == 'q':
        break
    second_number = input("Second number: ")
❶   try:
        answer = int(first_number) / int(second_number)
❷   except ZeroDivisionError:
        print("You can't divide by 0!")
❸   else:
        print(answer)
```

我们让Python尝试执行`try`代码块中的除法运算（见❶），这个代码块只包含可能导致错误的代码。依赖于`try`代码块成功执行的代码都放在`else`代码块中；在这个示例中，如果除法运算成功，我们就使用`else`代码块来打印结果（见❸）。

`except`代码块告诉Python，出现`ZeroDivisionError`异常时该怎么办（见❷）。如果`try`代码块

因除零错误而失败，我们就打印一条友好的消息，告诉用户如何避免这种错误。程序将继续运行，用户根本看不到**traceback**：

```
Give me two numbers, and I'll divide them.
Enter 'q' to quit.

First number: 5
Second number: 0
You can't divide by 0!

First number: 5
Second number: 2
2.5

First number: q
```

try-except-else代码块的工作原理大致如下：Python尝试执行**try**代码块中的代码；只有可能引发异常的代码才需要放在**try**语句中。有时候，有一些仅在**try**代码块成功执行时才需要运行的代码；这些代码应放在**else**代码块中。**except**代码块告诉Python，如果它尝试运行**try**代码块中的代码时引发了指定的异常，该怎么办。

通过预测可能发生错误的代码，可编写健壮的程序，它们即便面临无效数据或缺少资源，也能继续运行，从而能够抵御无意的用户错误和恶意的攻击。

10

10.3.5 处理 FileNotFoundError 异常

使用文件时，一种常见的问题是找不到文件：你要查找的文件可能在其他地方、文件名可能不正确或者这个文件根本就不存在。对于所有这些情形，都可使用**try-except**代码块以直观的方式进行处理。

我们来尝试读取一个不存在的文件。下面的程序尝试读取文件**alice.txt**的内容，但我没有将这个文件存储在**alice.py**所在的目录中：

```
alice.py

filename = 'alice.txt'

with open(filename) as f_obj:
    contents = f_obj.read()
```

Python无法读取不存在的文件，因此它引发一个异常：

```
Traceback (most recent call last):
  File "alice.py", line 3, in <module>
    with open(filename) as f_obj:
FileNotFoundError: [Errno 2] No such file or directory: 'alice.txt'
```

在上述**traceback**中，最后一行报告了**FileNotFoundError**异常，这是Python找不到要打开的文

件时创建的异常。在这个示例中，这个错误是函数open()导致的，因此要处理这个错误，必须将try语句放在包含open()的代码行之前：

```
filename = 'alice.txt'

try:
    with open(filename) as f_obj:
        contents = f_obj.read()
except FileNotFoundError:
    msg = "Sorry, the file " + filename + " does not exist."
    print(msg)
```

在这个示例中，try代码块引发FileNotFoundError异常，因此Python找出与该错误匹配的except代码块，并运行其中的代码。最终的结果是显示一条友好的错误消息，而不是traceback：

```
Sorry, the file alice.txt does not exist.
```

如果文件不存在，这个程序什么都不做，因此错误处理代码的意义不大。下面来扩展这个示例，看看在你使用多个文件时，异常处理可提供什么样的帮助。

10.3.6 分析文本

你可以分析包含整本书的文本文件。很多经典文学作品都是以简单文本文件的方式提供的，因为它们不受版权限制。本节使用的文本来自项目Gutenberg (<http://gutenberg.org/>)，这个项目提供了一系列不受版权限制的文学作品，如果你要在编程项目中使用文学文本，这是一个很不错的资源。

下面来提取童话*Alice in Wonderland*的文本，并尝试计算它包含多少个单词。我们将使用方法split()，它根据一个字符串创建一个单词列表。下面是对只包含童话名"Alice in Wonderland"的字符串调用方法split()的结果：

```
>>> title = "Alice in Wonderland"
>>> title.split()
['Alice', 'in', 'Wonderland']
```

方法split()以空格为分隔符将字符串分拆成多个部分，并将这些部分都存储到一个列表中。结果是一个包含字符串中所有单词的列表，虽然有些单词可能包含标点。为计算*Alice in Wonderland*包含多少个单词，我们将对整篇小说调用split()，再计算得到的列表包含多少个元素，从而确定整篇童话大致包含多少个单词：

```
filename = 'alice.txt'

try:
    with open(filename) as f_obj:
        contents = f_obj.read()
except FileNotFoundError:
```

```

    msg = "Sorry, the file " + filename + " does not exist."
    print(msg)
else:
    # 计算文件大致包含多少个单词
    ❶ words = contents.split()
    ❷ num_words = len(words)
    ❸ print("The file " + filename + " has about " + str(num_words) + " words.")

```

我们把文件alice.txt移到了正确的目录中，让try代码块能够成功地执行。在❶处，我们对变量contents（它现在是一个长长的字符串，包含童话*Alice in Wonderland*的全部文本）调用方法split()，以生成一个列表，其中包含这部童话中的所有单词。当我们使用len()来确定这个列表的长度时，就知道了原始字符串大致包含多少个单词（见❷）。在❸处，我们打印一条消息，指出文件包含多少个单词。这些代码都放在else代码块中，因为仅当try代码块成功执行时才执行它们。输出指出了文件alice.txt包含多少个单词：

```
The file alice.txt has about 29461 words.
```

这个数字有点大，因为这里使用的文本文件包含出版商提供的额外信息，但与童话*Alice in Wonderland*的长度相当一致。

10.3.7 使用多个文件

10

下面多分析几本书。这样做之前，我们先将这个程序的大部分代码移到一个名为count_words()的函数中，这样对多本书进行分析时将更容易：

word_count.py

```

def count_words(filename):
    ❶ """计算一个文件大致包含多少个单词"""
    try:
        with open(filename) as f_obj:
            contents = f_obj.read()
    except FileNotFoundError:
        msg = "Sorry, the file " + filename + " does not exist."
        print(msg)
    else:
        # 计算文件大致包含多少个单词
        words = contents.split()
        num_words = len(words)
        print("The file " + filename + " has about " + str(num_words) +
              " words.")

filename = 'alice.txt'
count_words(filename)

```

这些代码大都与原来一样，我们只是将它们移到了函数count_words()中，并增加了缩进量。修改程序的同时更新注释是个不错的习惯，因此我们将注释改成了文档字符串，并稍微调整了一

下措辞（见❶）。

现在可以编写一个简单的循环，计算要分析的任何文本包含多少个单词了。为此，我们将要分析的文件名称存储在一个列表中，然后对列表中的每个文件都调用count_words()。我们将尝试计算*Alice in Wonderland*、*Siddhartha*、*Moby Dick*和*Little Women*分别包含多少个单词，它们都不受版权限制。我故意没有将siddhartha.txt放到word_count.py所在的目录中，让你能够看到这个程序在文件不存在时处理得有多出色：

```
def count_words(filename):
    --snip--

filenames = ['alice.txt', 'siddhartha.txt', 'moby_dick.txt', 'little_women.txt']
for filename in filenames:
    count_words(filename)
```

文件siddhartha.txt不存在，但这丝毫不影响这个程序处理其他文件：

```
The file alice.txt has about 29461 words.
Sorry, the file siddhartha.txt does not exist.
The file moby_dick.txt has about 215136 words.
The file little_women.txt has about 189079 words.
```

在这个示例中，使用try-except代码块提供了两个重要的优点：避免让用户看到traceback；让程序能够继续分析能够找到的其他文件。如果不捕获因找不到siddhartha.txt而引发的FileNotFoundError异常，用户将看到完整的traceback，而程序将在尝试分析*Siddhartha*后停止运行——根本不分析*Moby Dick*和*Little Women*。

10.3.8 失败时一声不吭

在前一个示例中，我们告诉用户有一个文件找不到。但并非每次捕获到异常时都需要告诉用户，有时候你希望程序在发生异常时一声不吭，就像什么都没有发生一样继续运行。要让程序在失败时一声不吭，可像通常那样编写try代码块，但在except代码块中明确地告诉Python什么都不要做。Python有一个pass语句，可在代码块中使用它来让Python什么都不要做：

```
def count_words(filename):
    """计算一个文件大致包含多少个单词"""
    try:
        --snip--
    except FileNotFoundError:
        ❶ pass
    else:
        --snip--

filenames = ['alice.txt', 'siddhartha.txt', 'moby_dick.txt', 'little_women.txt']
for filename in filenames:
    count_words(filename)
```

相比于前一个程序，这个程序唯一不同的地方是❶处的`pass`语句。现在，出现`FileNotFoundError`异常时，将执行`except`代码块中的代码，但什么都不会发生。这种错误发生时，不会出现`traceback`，也没有任何输出。用户将看到存在的每个文件包含多少个单词，但没有任何迹象表明有一个文件未找到：

```
The file alice.txt has about 29461 words.
The file moby_dick.txt has about 215136 words.
The file little_women.txt has about 189079 words.
```

`pass`语句还充当了占位符，它提醒你在程序的某个地方什么都没有做，并且以后也许要在这里做点什么。例如，在这个程序中，我们可能决定将找不到的文件的名称写入到文件`missing_files.txt`中。用户看不到这个文件，但我们可以读取这个文件，进而处理所有文件找不到的问题。

10.3.9 决定报告哪些错误

在什么情况下该向用户报告错误？在什么情况下又应该在失败时一声不吭呢？如果用户知道要分析哪些文件，他们可能希望在有文件没有分析时出现一条消息，将其中的原因告诉他们。如果用户只想看到结果，而并不知道要分析哪些文件，可能就无需在有些文件不存在时告知他们。向用户显示他不想看到的信息可能会降低程序的可用性。Python的错误处理结构让你能够细致地控制与用户分享错误信息的程度，要分享多少信息由你决定。

编写得很好且经过详尽测试的代码不容易出现内部错误，如语法或逻辑错误，但只要程序依赖于外部因素，如用户输入、存在指定的文件、有网络链接，就有可能出现异常。凭借经验可判断该在程序的什么地方包含异常处理块，以及出现错误时该向用户提供多少相关的信息。

10

动手试一试

10-6 加法运算：提示用户提供数值输入时，常出现的一个问题是，用户提供的是文本而不是数字。在这种情况下，当你尝试将输入转换为整数时，将引发`TypeError`异常。编写一个程序，提示用户输入两个数字，再将它们相加并打印结果。在用户输入的任何值不是数字时都捕获`TypeError`异常，并打印一条友好的错误消息。对你编写的程序进行测试：先输入两个数字，再输入一些文本而不是数字。

10-7 加法计算器：将你认为完成练习 10-6 而编写的代码放在一个`while`循环中，让用户犯错（输入的是文本而不是数字）后能够继续输入数字。

10-8 猫和狗：创建两个文件`cats.txt`和`dogs.txt`，在第一个文件中至少存储三只猫的名字，在第二个文件中至少存储三条狗的名字。编写一个程序，尝试读取这些文件，并将其内容打印到屏幕上。将这些代码放在一个`try-except`代码块中，以便在文件不存在时捕获`FileNotFoundError`错误，并打印一条友好的消息。将其中一个文件移到另一个地

方，并确认 `except` 代码块中的代码将正确地执行。

10-9 沉默的猫和狗：修改你在练习 10-8 中编写的 `except` 代码块，让程序在文件不存在时一言不发。

10-10 常见单词：访问项目 Gutenberg (<http://gutenberg.org/>)，并找一些你想分析的图书。下载这些作品的文本文件或将浏览器中的原始文本复制到文本文件中。

你可以使用方法 `count()` 来确定特定的单词或短语在字符串中出现了多少次。例如，下面的代码计算 `'row'` 在一个字符串中出现了多少次：

```
>>> line = "Row, row, row your boat"
>>> line.count('row')
2
>>> line.lower().count('row')
3
```

请注意，通过使用 `lower()` 将字符串转换为小写，可捕捉要查找的单词出现的所有次数，而不管其大小写格式如何。

编写一个程序，它读取你在项目 Gutenberg 中获取的文件，并计算单词 `'the'` 在每个文件中分别出现了多少次。

10.4 存储数据

很多程序都要求用户输入某种信息，如让用户存储游戏首选项或提供要可视化的数据。不管专注的是是什么，程序都把用户提供的信息存储在列表和字典等数据结构中。用户关闭程序时，你几乎总是要保存他们提供的信息；一种简单的方式是使用模块 `json` 来存储数据。

模块 `json` 让你能够将简单的 Python 数据结构转储到文件中，并在程序再次运行时加载该文件中的数据。你还可以使用 `json` 在 Python 程序之间分享数据。更重要的是，JSON 数据格式并非 Python 专用的，这让你能够将以 JSON 格式存储的数据与使用其他编程语言的人分享。这是一种轻便格式，很有用，也易于学习。

注意 JSON (JavaScript Object Notation) 格式最初是为 JavaScript 开发的，但随后成了一种常见格式，被包括 Python 在内的众多语言采用。

10.4.1 使用 `json.dump()` 和 `json.load()`

我们来编写一个存储一组数字的简短程序，再编写一个将这些数字读取到内存中的程序。第一个程序将使用 `json.dump()` 来存储这组数字，而第二个程序将使用 `json.load()`。

函数 `json.dump()` 接受两个实参：要存储的数据以及可用于存储数据的文件对象。下面演示

了如何使用`json.dump()`来存储数字列表：

number_writer.py

```
import json

numbers = [2, 3, 5, 7, 11, 13]

❶ filename = 'numbers.json'
❷ with open(filename, 'w') as f_obj:
❸     json.dump(numbers, f_obj)
```

我们先导入模块`json`，再创建一个数字列表。在❶处，我们指定了要将该数字列表存储到其中的文件的名称。通常使用文件扩展名`.json`来指出文件存储的数据为JSON格式。接下来，我们以写入模式打开这个文件，让`json`能够将数据写入其中（见❷）。在❸处，我们使用函数`json.dump()`将数字列表存储到文件`numbers.json`中。

这个程序没有输出，但我们可以打开文件`numbers.json`，看看其内容。数据的存储格式与Python中一样：

```
[2, 3, 5, 7, 11, 13]
```

下面再编写一个程序，使用`json.load()`将这个列表读取到内存中：

number_reader.py

```
import json

❶ filename = 'numbers.json'
❷ with open(filename) as f_obj:
❸     numbers = json.load(f_obj)

print(numbers)
```

在❶处，我们确保读取的是前面写入的文件。这次我们以读取方式打开这个文件，因为Python只需读取这个文件（见❷）。在❸处，我们使用函数`json.load()`加载存储在`numbers.json`中的信息，并将其存储到变量`numbers`中。最后，我们打印恢复的数字列表，看看它是否与`number_writer.py`中创建的数字列表相同：

```
[2, 3, 5, 7, 11, 13]
```

这是一种在程序之间共享数据的简单方式。

10.4.2 保存和读取用户生成的数据

对于用户生成的数据，使用`json`保存它们大有裨益，因为如果不以某种方式进行存储，等程序停止运行时用户的信息将丢失。下面来看一个这样的例子：用户首次运行程序时被提示输入自

己的名字，这样再次运行程序时就记住他了。

我们先来存储用户的名字：

remember_me.py

```
import json

❶ username = input("What is your name? ")

filename = 'username.json'
with open(filename, 'w') as f_obj:
❷     json.dump(username, f_obj)
❸     print("We'll remember you when you come back, " + username + "!")
```

在❶处，我们提示输入用户名，并将其存储在一个变量中。接下来，我们调用`json.dump()`，并将用户名和一个文件对象传递给它，从而将用户名存储到文件中（见❷）。然后，我们打印一条消息，指出我们存储了他输入的信息（见❸）：

```
What is your name? Eric
We'll remember you when you come back, Eric!
```

现在再编写一个程序，向其名字被存储的用户发出问候：

greet_user.py

```
import json

filename = 'username.json'

with open(filename) as f_obj:
❶     username = json.load(f_obj)
❷     print("Welcome back, " + username + "!")
```

在❶处，我们使用`json.load()`将存储在`username.json`中的信息读取到变量`username`中。恢复用户名后，我们就可以欢迎用户回来了（见❷）：

```
Welcome back, Eric!
```

我们需要将这两个程序合并到一个程序（`remember_me.py`）中。这个程序运行时，我们将尝试从文件`username.json`中获取用户名，因此我们首先编写一个尝试恢复用户名的`try`代码块。如果这个文件不存在，我们就在`except`代码块中提示用户输入用户名，并将其存储在`username.json`中，以便程序再次运行时能够获取它：

remember_me.py

```
import json

# 如果以前存储了用户名，就加载它
```

```

# 否则,就提示用户输入用户名并存储它
filename = 'username.json'
try:
    ❶ with open(filename) as f_obj:
    ❷     username = json.load(f_obj)
    ❸ except FileNotFoundError:
    ❹     username = input("What is your name? ")
    ❺     with open(filename, 'w') as f_obj:
        json.dump(username, f_obj)
        print("We'll remember you when you come back, " + username + "!")
else:
    print("Welcome back, " + username + "!")

```

这里没有任何新代码,只是将前两个示例的代码合并到了一个程序中。在❶处,我们尝试打开文件username.json。如果这个文件存在,就将其中的用户名读取到内存中(见❷),再执行else代码块,即打印一条欢迎用户回来的消息。用户首次运行这个程序时,文件username.json不存在,将引发FileNotFoundError异常(见❸),因此Python将执行except代码块:提示用户输入其用户名(见❹),再使用json.dump()存储该用户名,并打印一句问候语(见❺)。

无论执行的是except代码块还是else代码块,都将显示用户名和合适的问候语。如果这个程序是首次运行,输出将如下:

```

What is your name? Eric
We'll remember you when you come back, Eric!

```

否则,输出将如下:

```

Welcome back, Eric!

```

这是程序之前至少运行了一次时的输出。

10.4.3 重构

你经常会遇到这样的情况:代码能够正确地运行,但可做进一步的改进——将代码划分为一系列完成具体工作的函数。这样的过程被称为重构。重构让代码更清晰、更易于理解、更容易扩展。

要重构remember_me.py,可将其大部分逻辑放到一个或多个函数中。remember_me.py的重点是问候用户,因此我们将其所有代码都放到一个名为greet_user()的函数中:

```

remember_me.py
import json

def greet_user():
    ❶ """问候用户,并指出其名字"""
    filename = 'username.json'
    try:

```

```

        with open(filename) as f_obj:
            username = json.load(f_obj)
    except FileNotFoundError:
        username = input("What is your name? ")
        with open(filename, 'w') as f_obj:
            json.dump(username, f_obj)
        print("We'll remember you when you come back, " + username + "!")
    else:
        print("Welcome back, " + username + "!")

```

`greet_user()`

考虑到现在使用了一个函数，我们删除了注释，转而使用一个文档字符串来指出程序是做什么的（见❶）。这个程序更清晰些，但函数`greet_user()`所做的不仅仅是问候用户，还在存储了用户名时获取它，而在没有存储用户名时提示用户输入一个。

下面来重构`greet_user()`，让它不执行这么多任务。为此，我们首先将获取存储的用户名的代码移到另一个函数中：

```

import json

def get_stored_username():
    ❶ """如果存储了用户名，就获取它"""
    filename = 'username.json'
    try:
        with open(filename) as f_obj:
            username = json.load(f_obj)
    except FileNotFoundError:
    ❷     return None
    else:
        return username

def greet_user():
    """问候用户，并指出其名字"""
    username = get_stored_username()
    ❸ if username:
        print("Welcome back, " + username + "!")
    else:
        username = input("What is your name? ")
        filename = 'username.json'
        with open(filename, 'w') as f_obj:
            json.dump(username, f_obj)
        print("We'll remember you when you come back, " + username + "!")

greet_user()

```

新增的函数`get_stored_username()`目标明确，❶处的文档字符串指出了这一点。如果存储了用户名，这个函数就获取并返回它；如果文件`username.json`不存在，这个函数就返回`None`（见❷）。这是一种不错的做法：函数要么返回预期的值，要么返回`None`；这让我们能够使用函数的返回值做简单测试。在❸处，如果成功地获取了用户名，就打印一条欢迎用户回来的消息，否则就提示

用户输入用户名。

我们还需将greet_user()中的另一个代码块提取出来：将没有存储用户名时提示用户输入的代码放在一个独立的函数中：

```
import json

def get_stored_username():
    """如果存储了用户名，就获取它"""
    --snip--

def get_new_username():
    """提示用户输入用户名"""
    username = input("What is your name? ")
    filename = 'username.json'
    with open(filename, 'w') as f_obj:
        json.dump(username, f_obj)
    return username

def greet_user():
    """问候用户，并指出其名字"""
    username = get_stored_username()
    if username:
        print("Welcome back, " + username + "!")
    else:
        username = get_new_username()
        print("We'll remember you when you come back, " + username + "!")

greet_user()
```

10

在remember_me.py的这个最终版本中，每个函数都执行单一而清晰的任务。我们调用greet_user()，它打印一条合适的消息：要么欢迎老用户回来，要么问候新用户。为此，它首先调用get_stored_username()，这个函数只负责获取存储的用户名（如果存储了的话），再在必要时调用get_new_username()，这个函数只负责获取并存储新用户的用户名。要编写出清晰而易于维护和扩展的代码，这种划分工作必不可少。

动手试一试

10-11 喜欢的数字：编写一个程序，提示用户输入他喜欢的数字，并使用json.dump()将这个数字存储到文件中。再编写一个程序，从文件中读取这个值，并打印消息“I know your favorite number! It's _____”。

10-12 记住喜欢的数字：将练习 10-11 中的两个程序合而为一。如果存储了用户喜欢的数字，就向用户显示它，否则提示用户输入他喜欢的数字并将其存储到文件中。运行这个程序两次，看看它是否像预期的那样工作。

10-13 验证用户：最后一个 remember_me.py 版本假设用户要么已输入其用户名，要

么是首次运行该程序。我们应修改这个程序，以应对这样的情形：当前和最后一次运行该程序的用户并非同一个人。

为此，在 `greet_user()` 中打印欢迎用户回来的消息前，先询问他用户名是否是对的。如果不对，就调用 `get_new_username()` 让用户输入正确的用户名。

10.5 小结

在本章中，你学习了：如何使用文件；如何一次性读取整个文件，以及如何以每次一行的方式读取文件的内容；如何写入文件，以及如何将文本附加到文件末尾；什么是异常以及如何处理程序可能引发的异常；如何存储Python数据结构，以保存用户提供的信息，避免用户每次运行程序时都需要重新提供。

在第11章中，你将学习高效的代码测试方式，这可帮助你确定代码正确无误，以及发现扩展现有程序时可能引入的bug。